

Developer Guide

Coupon Management & Invoice Generation Systems

TECHNICAL REFERENCE

Developer Guide — Coupon Management & Invoice Generation

A technical reference for how both systems are built and how data flows through them, from Mongoose schema → API route → helper libs → storefront/admin UI.

Audience: developers maintaining or extending this codebase.

Part 1 — Coupon Management System

1.1 Overview

Coupons are discount codes a shopper applies at checkout. The system spans:

- one Mongoose model (`Coupon`)
- one Zod validator
- four API routes (admin CRUD + public "available" + the pre-existing "apply")
- an admin management screen and a storefront "available offers" component

A key design rule runs through everything: **applying a coupon is free; only placing an order spends a use**. A shopper can add and remove a code at checkout as many times as they like without affecting `usedCount`.

1.2 Data model — `models/Coupon.ts`

Field	Type	Default	Meaning
<code>code</code>	String	—	Unique, uppercased, trimmed. The code the shopper types (e.g. <code>WELCOME10</code>).
<code>discountType</code>	<code>"percent" "flat"</code>	—	Percent of subtotal, or a flat rupee amount.
<code>value</code>	Number	—	<code>10</code> = 10% (percent) or ₹10 off (flat).
<code>minOrderValue</code>	Number	<code>0</code>	Subtotal must be ≥ this for the coupon to apply.
<code>expiresAt</code>	Date	—	The coupon is invalid at/after this instant.
<code>usageLimit</code>	Number	<code>0</code>	Max total redemptions across all users. <code>0</code> = unlimited.
<code>usedCount</code>	Number	<code>0</code>	How many times it has actually been redeemed (order placed).
<code>isActive</code>	Boolean	<code>true</code>	Master on/off switch.
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	From <code>timestamps: true</code> .

Constraints: `code` is `unique` (a duplicate insert throws Mongo error `11000`), and there's an explicit index on `code` for fast lookup at apply time.

1.3 Validation — `lib/validations/coupon.ts`

Two schemas share a common base:

- `couponCreateSchema` — all fields required (POST).
- `couponUpdateSchema` — `base.partial()` so any subset can be sent (PUT).

Both run a `superRefine` enforcing **percent discounts can't exceed 100**. `code` is uppercased via a Zod `.transform`, and `value` / `minOrderValue` / `usageLimit` use `z.coerce.number()` so string form inputs are accepted. `expiresAt` uses `z.coerce.date()`.

1.4 API routes

`GET /api/coupons` (admin) — list

`requireAdmin` → returns every coupon, newest first. Admin-only because it exposes usage limits and inactive/expired codes.

`POST /api/coupons` (admin) — create

`requireAdmin` → `couponCreateSchema` → pre-check for an existing code (409 if found) → `Coupon.create` → `logAdminAction("COUPON_CREATE", "Coupon", ...)`. A duplicate-key race is also caught and returned as 409.

GET /api/coupons/[id] (admin) — read one

PUT /api/coupons/[id] (admin) — update

`couponUpdateSchema` . If the code changes, uniqueness is re-checked against *other* documents. Logs `COUPON_UPDATE` .

DELETE /api/coupons/[id] (admin) — delete

Logs `COUPON_DELETE` .

GET /api/coupons/available (public) — storefront offers

No auth. Returns only coupons the shopper can actually use *right now*:

```
{ isActive: true,
  expiresAt: { $gt: now },
  $expr: { $or: [ { $eq: ["$usageLimit", 0] },
    { $lt: ["$usedCount", "$usageLimit"] } ] } }
```

`.select("code discountType value minOrderValue expiresAt")` — no `usedCount` or limits leak to the client. Sorted by `minOrderValue` ascending.

POST /api/coupons/apply (auth) — validate against a cart

Pre-existing route. Given `{ code, subtotal }` , it validates (active, not expired, under usage limit, subtotal $\geq$ min) and returns `{ code, discount }` . **It does not touch `usedCount`** . Discount math:

```
discount = discountType === "percent"
  ? Math.round((subtotal * value) / 100)
  : Math.min(value, subtotal); // flat can't exceed the subtotal
```

All new routes set `export const dynamic = "force-dynamic"` so Next never statically freezes them (a GET frozen at build time would 405 on other methods).

1.5 Where a use is actually "spent"

Redemption is re-validated server-side at order time (never trusting the client's earlier apply response, since the cart may have changed) and `usedCount` is incremented — but **the timing differs by payment method**, which is important to understand:

- **COD** — `app/api/orders/route.ts` : on order creation it re-validates the coupon and, if valid, does `coupon.usedCount += 1`; `await coupon.save()` right then. COD orders are considered placed immediately.
- **Razorpay** — `app/api/payments/razorpay/create-order/route.ts` validates the coupon and stores `couponCode` on the (pending) order **but does not increment**. The increment happens only after payment succeeds, in `lib/confirmRazorpayPayment.ts` :

```
js if (order.couponCode) { await Coupon.updateOne({ code: order.couponCode }, { $inc: { usedCount: 1 } }); }
```

Rationale: a coupon shouldn't be spent on a checkout the shopper abandons before paying.

Extension note: `usageLimit` is a global cap. There is currently no per-user limit. To add "one use per customer", create a `CouponRedemption` collection keyed on `{coupon, user}` (unique) and check it in the apply route and both order paths.

1.6 Storefront usage

`components/storefront/AvailableCoupons.tsx` (client) fetches `/api/coupons/available` , renders each offer as a dashed chip with a one-tap **Apply** button, and: - greys out coupons whose `minOrderValue` exceeds the current `subtotal` ("Add ₹X more to use"), - shows "Applied" when a code matches the currently applied one, - renders **nothing** when there are no available coupons (safe to drop anywhere).

Props: `subtotal?` , `appliedCode?` , `onApply?(code)` . At checkout it's wired to `onApply={(c) => handleApplyCoupon(c)}` — `handleApplyCoupon` was refactored to take an optional code so a tapped chip applies directly.

1.7 Admin usage

`app/admin/coupons/page.tsx` (client) — a create/edit form plus a table showing live `usedCount / limit`, expiry, and a computed status pill (Active / Inactive / Expired / Used up). Clicking the pill toggles `isActive` via PUT. The form `PUT`s when editing an existing coupon and `POST`s when creating.

1.8 Coupon data flow (end to end)

```
Shopper at checkout
├ AvailableCoupons → GET /api/coupons/available (public, safe fields)
└ taps Apply / types code → POST /api/coupons/apply {code, subtotal}
  → returns {discount} (usedCount NOT touched)
|
Place order
├ COD: POST /api/orders → re-validate → usedCount += 1 (immediately)
└ Razorpay: POST .../create-order → re-validate, store couponCode (pending)
  → payment success → confirmRazorpayPayment → $inc usedCount

Admin
└ /admin/coupons → GET/POST/PUT/DELETE /api/coupons/{[id]} → audit log
```

Part 2 — Invoice Generation System

2.1 Overview

Every order gets a GST-compliant tax-invoice PDF, generated by a **zero-dependency** PDF engine (no npm packages, no font files — uses the PDF standard-14 Helvetica fonts). The invoice is *issued* (numbered + persisted as an immutable snapshot) the first time it's needed — at order-confirmation email time or first download — and re-rendered from that snapshot forever after.

Files:

```
models/InvoiceSettings.ts    singleton CMS config
models/Invoice.ts           issued-invoice record (snapshot)
lib/validations/invoiceSettings.ts  Zod schema
lib/invoice/settings.ts      defaults, loader, financial-year helper
lib/invoice/pdf.ts          low-level PDF writer (PdfDoc class)
lib/invoice/compute.ts       GST math, numbering, getOrCreateInvoice()
lib/invoice/render.ts        draws the document (snapshot → PDF bytes)
lib/invoice/email.ts         order email WITH the PDF attached
app/api/invoice-settings/route.ts  GET/PUT config (admin)
app/api/invoices/route.ts        invoice register list (admin)
app/api/invoices/[orderId]/route.ts  the PDF (owner or admin)
app/admin/invoices/page.tsx      admin CMS + register
```

2.2 The two models

InvoiceSettings (singleton, key **"invoice"**)

Admin-editable configuration, grouped:

- seller** — legalName, tradeName, address (line1/line2/city/state/**stateCode**/pincode/country), **gstIn**, **pan**, **cin**, email, phone, website, logoUrl.
- tax** — **gstEnabled**, **defaultGstRate** (e.g. 18), **defaultHsnCode**, **pricesIncludeTax** (default true), **shippingHsnCode** (996812), **shippingGstRate**, **roundOffTotal**.
- numbering** — **prefix** (INV), **includeFinancialYear**, **padding** (5 → **00042**), **nextSequence**, **sequenceFinancialYear**, **resetEachFinancialYear**.
- document** — **title**, **declaration**, **terms**, **notes**, **footerText**, **signatureLabel**, **showBankDetails**, **showHsnSummary**, **showAmountInWords**.
- bank** — accountName, accountNumber, bankName, ifsc, branch.
- attachToOrderEmail** — attach the PDF to order emails.

Invoice (one per order — **order** is unique)

Field	Meaning
order , user	refs; order unique = one invoice per order
invoiceNumber	e.g. INV/2026-27/00042 (unique)
financialYear , sequence , issuedAt	numbering metadata
grandTotal , totalTax , buyerName , placeOfSupply , isInterState	denormalized for fast admin listing
snapshot	Mixed — the frozen, fully-computed document (see InvoiceSnapshot)

Why a snapshot? An invoice is a fixed legal document. If you later change the GST rate, your address, or a product price, previously issued invoices must still render exactly as issued. Re-deriving them would rewrite financial history, so we freeze everything printed at issue time.

2.3 Settings loader — `lib/invoice/settings.ts`

- `getInvoiceSettings()` — reads the singleton (creates it with `DEFAULT_INVOICE_SETTINGS` on first use), deep-merges stored values over the defaults (reusing `mergeSettings` from `lib/site-settings.ts`), and falls back to defaults on any DB error so rendering never hard-fails.
- `financialYearOf(date)` — Indian FY (Apr 1 → Mar 31). `2026-07-21 → "2026-27"`.

2.4 The PDF engine — `lib/invoice/pdf.ts`

A `PdfDoc` class that writes raw PDF. Top-left origin with y growing downward (CSS-like) for sane layout. Key methods: `text()`, `line()`, `rect()`, `textWidth()`, `truncate()`, `wrap()`, `addPage()`, `build(): Buffer`.

Because standard-14 fonts use WinAnsi encoding (no ₹ glyph), `sanitize()` transliterates `₹ → "Rs."` and normalizes smart quotes/dashes. The on-screen HTML keeps the real ₹.

2.5 The GST engine — `lib/invoice/compute.ts`

This is the heart of the system. Key exports:

- `stateCodeFor(stateName)` — maps a state name to its 2-digit GST code.
- `amountInWords(amount)` — Indian format (lakh/crore), with paise.
- `computeInvoiceSnapshot(order, settings, buyer, meta)` — pure function, `order → InvoiceSnapshot`. No DB, fully testable.
- `getOrCreateInvoice(orderId)` — the orchestrator (below).

Tax logic inside `computeInvoiceSnapshot`:

1. **Intra vs inter-state.** Compare seller `stateCode` with the buyer's shipping state code. Same → CGST + SGST (half rate each). Different → IGST (full rate). If either code is unknown, it falls back to intra-state (the safer default).
2. **Discount apportionment.** The order-level discount is split across line items in proportion to each line's gross value; the **last line absorbs the rounding remainder** so the parts sum to the order discount exactly.
3. **Tax extraction vs addition.** If `pricesIncludeTax` (default), tax is pulled out of the price: `taxable = net × 100 / (100 + rate)`. Otherwise it's added on top.
4. **Equal halves.** CGST and SGST are each `round2(tax / 2)` — they always print as identical amounts (a one-paisa gap is an audit red flag). Sub-rupee drift is absorbed by the round-off line.
5. **Shipping** is treated as its own taxable supply (its own HSN + rate).
6. **HSN/SAC summary** groups lines by `(hsn, rate)` for the rate-wise tax table.
7. **Round-off.** If `roundOffTotal`, the grand total rounds to the nearest rupee and a round-off line shows the difference.

`InvoiceSnapshot.totals` includes `gross`, `discount`, `taxableValue`, `cgst`, `sgst`, `igst`, `totalTax`, `shippingFee/Taxable/Tax`, `beforeRounding`, `roundOff`, `grandTotal`, and `amountCharged` (the order's actual total, for reconciliation).

Numbering — `allocateInvoiceNumber (atomic)`. Uses a single `findOneAndUpdate` with `$inc` on `numbering.nextSequence`, so two concurrent orders can never get the same number. When `resetEachFinancialYear` is on and the FY rolled over, the counter resets to 1 in the same atomic write.

`getOrCreateInvoice(orderId)` — returns the existing invoice if present; otherwise loads the order + settings + buyer, allocates a number, builds the snapshot, and `Invoice.create` s it. A duplicate-key race (unique `order`) is caught and resolved by re-reading the winner. **Idempotent** — always one invoice per order.

2.6 Rendering — `lib/invoice/render.ts`

`renderInvoicePdf(snapshot): Buffer` lays out: title bar → seller + invoice meta → Bill To / Ship To → line-item table (HSN, qty, rate, taxable, GST%, tax, amount) → totals (CGST+SGST or IGST, shipping, round-off, grand total) → amount in words → HSN/SAC summary → bank/terms/declaration/signature → footer. It **auto-paginates** long orders, repeating the table header on each page. The column set adapts when `gstEnabled` is false (tax columns disappear).

2.7 Email attachment — `lib/invoice/email.ts`

`sendOrderEmailWithInvoice({ orderId, to, subject, html })` builds its own nodemailer transport, and if `attachToOrderEmail` is on, calls `getOrCreateInvoice` + `renderInvoicePdf` and attaches the PDF. Deliberately kept **separate** from the app's core `lib/`

`email.ts` (whose `{to,subject,html}` contract is used everywhere) so invoicing can't regress transactional email. Failures are swallowed — an email/PDF problem never blocks an order.

2.8 API routes

- `GET /api/invoice-settings` (admin) / `PUT` (admin) — read/replace config. PUT is Zod-validated, logs to the Activity Log, and **clamps `nextSequence` so it can be raised but never lowered** (reusing a number = duplicate legal doc).
- `GET /api/invoices` (admin) — the invoice register: search by number/name, date range, pagination, plus aggregate `grandTotal` / `totalTax` across the whole filtered set (for GST filing).
- `GET /api/invoices/{orderId}` (owner or admin) — issues on first call, then streams the PDF. `?download=1` forces a save dialog, otherwise inline.

2.9 Admin & storefront usage

- **Admin** `app/admin/invoices/page.tsx` — tabbed CMS (Seller & GSTIN / Tax / Numbering / Document / Bank) plus an **Invoice Register** tab with totals and view/download links.
- **Customer** order page has a **Download Invoice** button → `/api/invoices/{orderId}?download=1`.
- **Admin orders** list has a per-order **Invoice** link.

2.10 Invoice data flow (end to end)

```
Order confirmed (COD create, or Razorpay payment success)
├─ sendOrderEmailWithInvoice(orderId, ...)
│   └─ getInvoiceSettings() → attachToOrderEmail?
│       └─ getOrCreateInvoice(orderId)
│           ├── allocateInvoiceNumber() (atomic $inc)
│           ├── computeInvoiceSnapshot() (GST math, pure)
│           └─ Invoice.create({ snapshot, ... }) ← frozen here
└─ renderInvoicePdf(snapshot) → attach → send

Later download (customer or admin)
├─ GET /api/invoices/{orderId}
└─ getOrCreateInvoice() → returns existing → renderInvoicePdf(snapshot)

Admin config
├─ /admin/invoices → GET/PUT /api/invoice-settings → audit log
└─ Register tab → GET /api/invoices (search/date/totals)
```

2.11 Field reference — the tax math in one place

Given a line with gross `G`, allocated discount `d`, net `N = G - d`, rate `r%`:

Setting	Taxable	Tax
<code>pricesIncludeTax = true</code>	<code>N × 100 / (100 + r)</code>	<code>N - taxable</code>
<code>pricesIncludeTax = false</code>	<code>N</code>	<code>N × r / 100</code>
intra-state	—	<code>CGST = SGST = tax / 2</code>
inter-state	—	<code>IGST = tax</code>

Grand total = `round(Σ taxable + Σ tax)` when `roundOffTotal`, else unrounded. `amountCharged` (order total) is stored alongside for reconciliation — with `pricesIncludeTax` on, the two match to the rupee.

2.12 Extending the invoice system

- **Per-product HSN & GST rate.** Today one `defaultHsnCode` / `defaultGstRate` applies to all lines. Add `hsnCode` and `gstRate` to the `Product` model, carry them onto the order item at checkout, and read them per line in `computeInvoiceSnapshot` (fall back to the defaults when absent).

- **Credit notes** for refunds/cancellations — reuse `PdfDoc` + a variant of the snapshot with a `CREDIT NOTE` title and negative amounts.
 - **e-Invoice / IRN (GSTN)** — the snapshot already holds every field the IRP needs; add a post-issue step that submits it and stores the returned IRN/QR.
-

Appendix — audit actions used

Coupons: `COUPON_CREATE` , `COUPON_UPDATE` , `COUPON_DELETE` . Invoices: settings changes log `SETTINGS_UPDATE` with `changes.scope = "invoice"` . All were already present in `models/AuditLog.ts` .